

Chapter 3 - The quality of software

1. Software testing
2. Software quality factors
3. Metrics



1

1 - Testing the software



2

Introduction



- Objective of the test
 - Minimize the chances of an **anomaly** appearing when using software
 - Detecting software **defects**
- Note
 - The test does **not allow** the conclusion that a program is **free of defects**

Test levels (1)



- **Unit Testing**
 - Allows you to check that the functions (or methods) of a software program are working properly
 - Allows you to check the **adequacy** between the function and its specifications

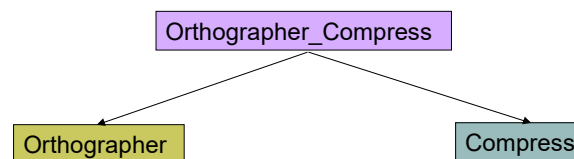
```
Effects: Calculates the factorial of a number
Inputs: n : a positive or zero integer
Outputs: the factorial of the number n
Function factorial(n: integer): integer
Variables
  f, i: integer
Begin
  i <- n
  While i <> 1 do
    f <- f * i
    i <- i - 1
  End
  Return f
End
```

Test levels (2)



- **Integration Testing**

- Software is made up of **several modules** (subsystems) that communicate with each other
- Allows you to test the **interactions** between modules that are being integrated
- It is assumed that the modules have been tested **separately** and are working properly



Cours de Génie Logiciel - Michel Ianotto

5

5

Test Levels (3)



- **Test, validation or acceptance**

- Helps verify that the software **is in line** with the customer's needs
- Allows you to verify that the software performs **all the functionalities** expressed in the specifications
- Example: Orthographe_Compresser
 - Set the file compression ratio

Cours de Génie Logiciel - Michel Ianotto

6

6

Non-regression tests

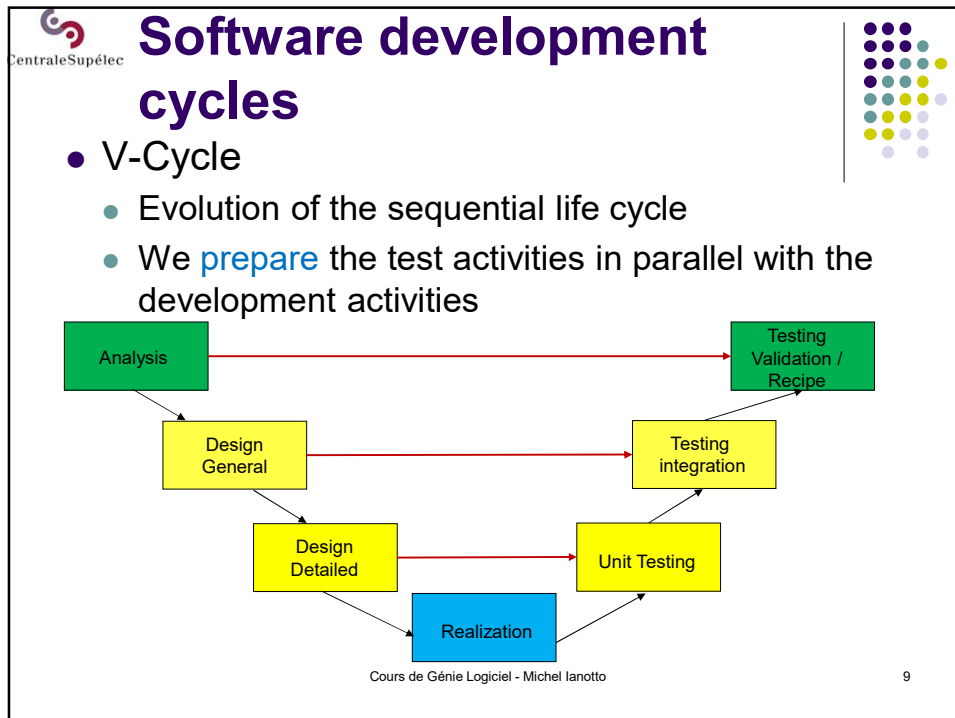


- A **non-regression test** consists of rerunning existing tests after a **change** in the software:
 - Verify that features that have already been validated are still working correctly.
 - Before modification:
 - feature A → OK
 - feature B → OK
 - feature C → OK (change)
 - After modification:
 - feature A → OK
 - feature B → OK
 - feature C → OK ← Risk of Regression

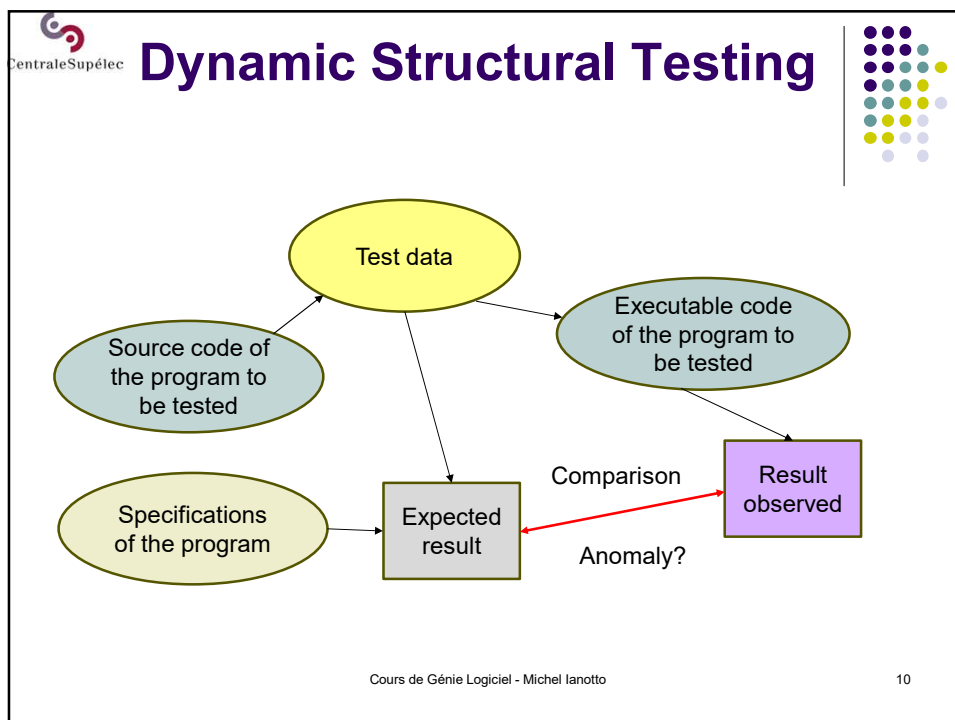
Software Development Cycles - Reminder



- The purpose of the **software development cycle** is to explain how the different stages of software development are **linked together**.
- Major development cycles
 - Sequential cycle
 - Iterative cycle
 - V-cycle



9



10

Dynamic structural test: control graph coverage method (1)



1. Represent the **structure of the program** in the form of a graph
 - **Control Graph**
2. Setting a criterion for the coverage of the control graph
 - Pass through all the vertices (edges) of the graph **at least once**
3. Produce the test data to meet this criterion

Dynamic structural test: control graph coverage method (2)



4. The **program is run** with the test data
5. Observed results are **compared** to expected results
6. Coverage rate **of all vertices**:

$$Ter1 = c1 = \frac{\text{number of covered vertices}}{\text{total number of vertices}}$$

4. Coverage rate **of all arcs**:

$$Ter2 = c2 = \frac{\text{number of covered edges}}{\text{total number of edges}}$$

Dynamic structural test: control graph coverage method (3)



Effects: Performs the calculation of $2 * x + 1$ if $x \neq 0$.
If $x = 0$, the function returns 0.

Inputs:

x : an integer

Outputs: $2 * x + 1$ if $x \neq 0$, returns 0 if $x = 0$

Function $f(x : \text{integer}) : \text{integer}$

Variables

$y : \text{integer}$

Begin

If $x \neq 0$ **then**

$x \leftarrow 2 * x$

EndIf

$y \leftarrow x + 1$

Return y

End

Cours de Génie Logiciel - Michel Ianotto

13

13

Dynamic structural test: control graph coverage method (4)



- Limitations of the method

a	a	S
Even	Even	0
Even	Odd	1
Odd	Even	2
Odd	Odd	3

Read (a)

Read (b)

X = a % 2

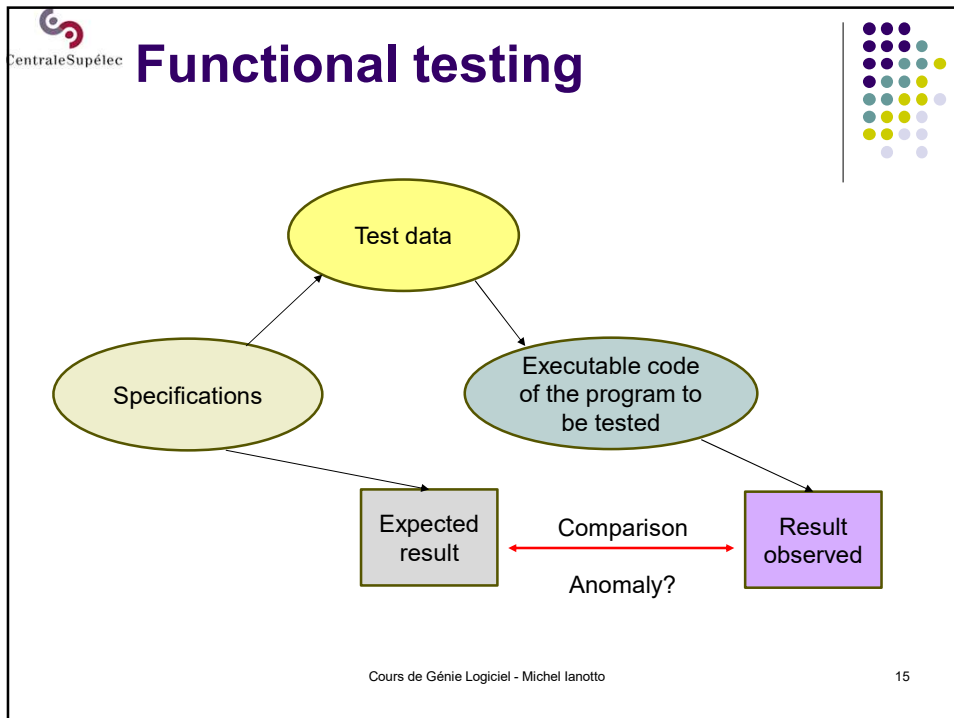
Y = b % 2

S = x + y

Cours de Génie Logiciel - Michel Ianotto

14

14



15

CentraleSupélec

Functional testing: partitional analysis method (1)

Effects: Searches for an element in an array where the elements are sorted in ascending order.

Inputs:
 key: an integer
 Array: A sorted array

Outputs: the index of the element if it is present in the table, -1 otherwise.

```

Function search(key: integer, array[]: integer): integer
Variables
  begin, end, middle, result: integer
  found: boolean
Begin
  start <- 0
  end <- array.length - 1
  result <- -1
  found <- False
  
```

Cours de Génie Logiciel - Michel Ianotto

16

16

Functional testing: partitional analysis method (2)



```
While (start <= end) And (found = False) do
  middle = (start + end) / 2;
  If (array[middle] = key) then
    result <- middle;
    found = True
  Else
    If (array[middle] < key) then
      start <- middle + 1
    Else
      end <- middle - 1
    FinSi
  FinSi
EndWhile
Return result
End
```

Functional testing: partitional analysis method (3)



- “Black Box” Test Methods
- Less used test method than dynamic structural testing:
 - It's not easy to automatically produce test data from **natural language specifications**
 - It is not always easy to **identify categories** of data of the same nature

Static structural testing: data flow analysis method (1)



```
Procedure f()  
Variables  
  a, b, c, x, y, z: integer  
Begin  
  read(c)  
  x <- 7  
  y <- x + a  
  b <- x + y + a  
  c <- y + 2 * x + z  
  write(s)  
  write(c)  
End
```

Cours de Génie Logiciel - Michel Ianotto

19

19

Static structural testing: data flow analysis method (2)



- Simple to implement
- The rule $\pi(x) = r....$ is used during the compilation phase of a program
- A method that works for functions comprising a sequential sequence of instructions

Cours de Génie Logiciel - Michel Ianotto

20

20

Google Test



- Google Test (or GTest) is a **unit testing framework** for C++, developed by Google.
- It allows you to write **automated tests** to verify that a function or class produces the **expected result**.
- The framework takes care of:
 - run the tests,
 - to determine who succeeds or fails,
 - display errors,
 - count tests passed and failed.

Cours de Génie Logiciel - Michel Ianotto

21

21

Google Test : *example (1)*



```
Calculator.hpp
#pragma once

class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    int subtract(int a, int b) {
        return a - b;
    }
};
```

Cours de Génie Logiciel - Michel Ianotto

22

22

Google Test : example (2)

```
CalculatorTest.cpp

#include <gtest/gtest.h>
#include "../src/Calculator.hpp"
// In the CalculatriceTest test suite,
// we define a test called Addition.
TEST(CalculatorTest, Addition) {
    // Preparation
    Calculator calc;
    // Calling the code under test
    int result = calc.add(2, 3);
    // Verification
    EXPECT_EQ(result, 5);
}

TEST(CalculatorTest, Subtraction) {
    Calculator calc;

    EXPECT_EQ(calc.subtract(5, 3), 2);
}
```

Cours de Génie Logiciel - Michel Ianotto

23

23

Google Test : example 1 (3)

- Project organization

```
Calculator/
├─ Makefile
├─ src/
│   └─ Calculator.hpp
└─ tests/
    └─ CalculatorTest.cpp
```

Cours de Génie Logiciel - Michel Ianotto

24

24

Google Test : exemple 1 (4)



- Compilation et exécution

```
prompt> make
```

```
prompt> calculatriceTest
```

- Résultat :

```
[=====] Running 2 tests from 1 test suite.  
[-----] 2 tests from CalculatriceTest  
[ RUN      ] CalculatriceTest.Addition  
[         OK ] CalculatriceTest.Addition  
[ RUN      ] CalculatriceTest.Soustraction  
[         OK ] CalculatriceTest.Soustraction  
[-----] 2 tests from CalculatriceTest  
[=====] 2 tests from 1 test suite ran.  
[ PASSED   ] 2 tests.
```

Google Test



- Assertions

- `EXPECT_EQ`

Verify that two values are equal:

```
EXPECT_EQ (result, 5);
```

- `EXPECT_NE`

Check that they are different:

```
EXPECT_NE (result, 6);
```

- `EXPECT_TRUE`

```
EXPECT_TRUE (condition);
```

- `EXPECT_FALSE`

```
EXPECT_FALSE (condition);
```

2 – Quality factors



27

Software quality factors (1)



- Software **that works** is not necessarily **quality** software.
- An application can implement **all the functionalities** expressed in the specifications, but:
 - be **extremely slow**,
 - be **difficult to use**,
 - be very difficult **to change**,
 - contain a lot of **duplicate code**.
 - be vulnerable to **attack**,
 - only work on a **particular configuration**.

28

Software quality factors (2)



- The **quality** of software therefore depends on **several factors**.
- Software quality **factors**: **attributes** that characterize software.

Software quality factors (3)



- **Ergonomics** → **External** quality factor
 - **Reusability**
 - Ability of software to reuse source code in whole or in part in new applications
 - **Maintainability**
 - Ease of correcting or upgrading software
 - **Readability (comprehensibility):**
 - Ease of understanding of the source code of a software
- } **Internal** quality factors

Software quality factors (4)



- The quality factors expected of a software must be **explicitly** described at the start of the project
 - Describe the **methods** used to calculate or evaluate them
 - What are the **expected** values
- At the end of the project, we check whether the **set values** have been **reached**.
- Word processing software
 - **Ergonomics**, **performance** and **maintainability**.

Cours de Génie Logiciel - Michel Ianotto

31

31

3 – Metrics



Cours de Génie Logiciel - Michel Ianotto

32

32

Metrics



- Prediction metrics
 - Numerical values that can be calculated from software
 - Metrics that can be used to assess quality factors (model)
 - Metrics that can be used as indicators of certain internal quality factors (readability, maintainability, etc.)

Prediction metrics (1)



- Examples of metrics that can be used to evaluate the readability of software (or used as an indicator)
- Nesting depth of control statements:
if... then ... if not OR As long as ... do

```

If a > 0 then
  While a > 1 do
    While j > 1 do
      If j = 2 * x then
        y = j
      Else
        j = d + 1
      EndIf
    EndWhile
  EndWhile
End If
  
```

Nesting depth
high: Low readability

Prediction metrics (2)



- Average length of program identifiers
 - Long identifiers make it easier to read a program
`int nbExemplarAvailable` instead of `int nED`
- Ratio of number of lines of comments to number of lines of instructions
 - Low ratio
 - Lack of comments
 - Low readability

Prediction Metrics (3)

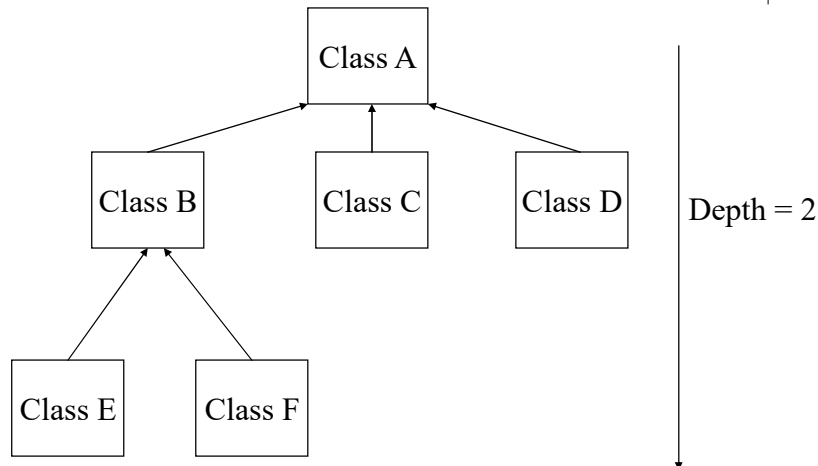


- Average number of lines of code per module (file)
- Code duplication: The amount of duplicate code :
$$\text{DuplicationRate} = \frac{\text{duplicate lines}}{\text{totale lines}} \times 100$$
 - High duplication can make the code difficult to read.

Object-oriented metrics (1)



- Depth of the Inheritance Tree



Cours de Génie Logiciel - Michel Ianotto

37

37

Object-oriented metrics (2)



- A class that corresponds to the leaves of the inheritance tree has a behavior that is **difficult** to understand:
 - To understand a class, you have to understand the super-classes.
 - The deeper a class is in the inheritance tree, the **greater** the number of attributes and methods it inherits.
 - This makes it less easy for the class to understand.

Cours de Génie Logiciel - Michel Ianotto

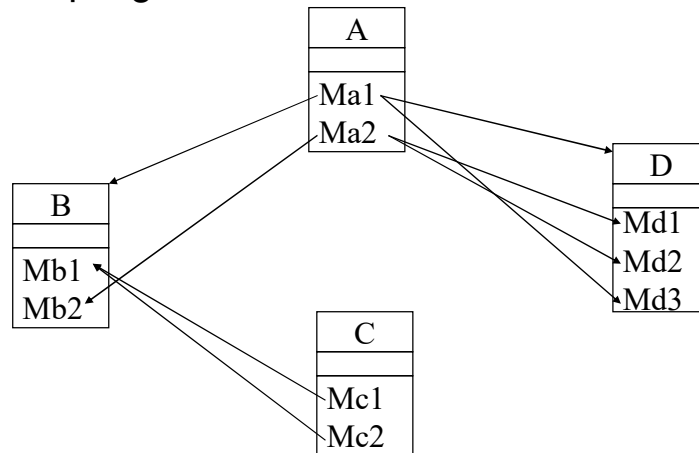
38

38

Object-oriented metrics (3)



- Coupling between classes



Cours de Génie Logiciel - Michel Ianotto

39

39

Object-oriented metrics (4)



Classes	Coupling metric
A	2
B	2
C	1
D	1

- Classroom coupled with many other classes
 - **Changing** the code of the class can cause the code of other classes to be changed
 - **Decreased** maintainability

Cours de Génie Logiciel - Michel Ianotto

40

40



Relation entre facteur de qualité et métriques (1)



- Difficulty in assessing a quality factor
 - **Mathematical relationship** (model) between quality factor and metrics is not always known.
- In practice :
 - Define metrics that will be used as an **indicator** of a quality factor.
 - Calculate metrics that are related to a quality factor
 - **Compare** these values to standard values obtained from past software development experiences.

Cours de Génie Logiciel - Michel Ianotto

41

41



Relation entre facteur de qualité et métriques (2)



- Quality factor: **maintainability** of software

Metric	Software studied	Reference
Medium cyclomatic complexity	8,2	4,5
Average coupling between classes	6,1	3,2
Depth of inheritance	4,3	3,0
Lines of code per file	28	18
Test coverage	62 %	85 %

Cours de Génie Logiciel - Michel Ianotto

42

42